TP 3 : Pipeline Jenkins CI/CD

Jenkins · SonarQube · Bandit · Pylint · Docker · Quality Gate

Objectif du TP

À l'issue de ce TP, vous aurez configuré un pipeline Jenkins complet (9 stages) pour ShopFlow : lint, tests unitaires, tests d'intégration, coverage avec seuil bloquant, analyse statique SonarQube avec Quality Gate, scan sécurité Bandit, et déploiement staging automatisé sur la branche main.

Stack	Jenkins · Docker · SonarQube · Bandit · Pylint · Flake8 · pytest · Coverage.py
Prérequis	TP1 (tests unitaires) + TP2 (tests intégration) complétés et fonctionnels

Partie 1 : Mise en place de l'environnement CI

Dans cette partie, vous allez démarrer Jenkins et SonarQube via Docker Compose, les configurer pour ShopFlow, et vérifier que les outils communiquent correctement.

1.1 Stack CI/CD avec Docker Compose

Créez le fichier `docker-compose.ci.yml` à la racine du projet ShopFlow :

```
docker-compose.ci.yml
# docker-compose.ci.yml — Stack CI/CD locale
# docker-compose.ci.yml
version: '3.9'
services:

  jenkins:
    image: jenkins/jenkins:lts-jdk17
    container_name: shopflow-jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    restart: unless-stopped

  sonarqube:
    image: sonarqube:community
    container_name: shopflow-sonarqube
    ports:
      - "9000:9000"
    environment:
      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
    volumes:
      - sonar_data:/opt/sonarqube/data
      - sonar_logs:/opt/sonarqube/logs
    restart: unless-stopped

volumes:
  jenkins_home:
  sonar_data:
  sonar_logs:
```

```
Terminal
# Prérequis Linux/Mac — requis pour SonarQube/ElasticSearch
sudo sysctl -w vm.max_map_count=262144

# Démarrer la stack CI
docker compose -f docker-compose.ci.yml up -d

# Vérifier que les deux services sont Up
docker compose -f docker-compose.ci.yml ps

# Récupérer le mot de passe initial Jenkins
docker exec shopflow-jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

URLs d'accès après démarrage

- http://localhost:8080 → Jenkins (mot de passe initial : voir commande ci-dessus)
- http://localhost:9000 → SonarQube (login : admin / admin changer au premier accès)
- Attendre le démarrage SonarQube (démarrage ElasticSearch) et Jenkins
- Vérifier : Jenkins → Manage Jenkins → System Info doit afficher Java 17

1.2 Configuration Jenkins

Depuis l'interface Jenkins (`http://localhost:8080`), effectuez les configurations suivantes :

Plugins à installer

Jenkins → Manage Jenkins → Plugins → Available plugins — recherchez et installez :

```
Plugins requis
Pipeline                    # Jenkinsfile DSL
SonarQube Scanner           # intégration SonarQube
HTML Publisher              # rapport coverage HTML
JUnit                       # rapport tests XML
Docker Pipeline             # agents Docker
Git                         # checkout SCM (souvent déjà installé)
```

Configurer SonarQube dans Jenkins

```
Configuration Jenkins ↔ SonarQube
Jenkins → Manage Jenkins → System → SonarQube Servers
  Name     : SonarQube
  URL      : http://sonarqube:9000   ← nom Docker, pas localhost !
  Token    : (créer dans SonarQube puis ajouter comme Credential)

# Créer le token SonarQube :
# SonarQube → Admin → My Account → Security → Generate Token
# Jenkins → Manage Credentials → Global → Add → Secret text
# ID : sonar-token
```

1.3 Fichier sonar-project.properties

Vérifiez que ce fichier existe à la racine de ShopFlow (créé en TP0) :

```
sonar-project.properties
# sonar-project.properties
sonar.projectKey=shopflow
sonar.projectName=ShopFlow API
sonar.projectVersion=0.1.0
sonar.sources=app
sonar.tests=tests
sonar.language=python
sonar.python.coverage.reportPaths=coverage.xml
sonar.python.pylint.reportPaths=pylint-report.txt
sonar.python.xunit.reportPath=junit-report.xml
sonar.exclusions=**/migrations/**,**/__init__.py
sonar.qualitygate.wait=true
```

Q1 : Environnement CI

1.1. Démarrez la stack CI. Copiez la sortie de docker compose -f docker-compose.ci.yml ps montrant les deux services Up.

1.2. Accédez à Jenkins (http://localhost:8080). Installez les plugins listés.

1.3. Configurez SonarQube dans Jenkins. Pourquoi utilise-t-on l'URL http://sonarqube:9000 plutôt que http://localhost:9000 ?

1.4. Créez un projet ShopFlow dans SonarQube (Administration → Projects → Create). Notez la clé de projet utilisée.

Partie 2 : Jenkinsfile : Stages Lint, Tests et Coverage

Vous allez écrire le Jenkinsfile en plusieurs étapes. Commencez par les stages fondamentaux : installation des dépendances, lint, tests unitaires, tests d'intégration et coverage.

2.1 Structure du Jenkinsfile

Créez le fichier `Jenkinsfile` à la racine du projet ShopFlow :

Jenkinsfile — squelette de base

```
// Jenkinsfile — ShopFlow CI/CD Pipeline
pipeline {
    agent {
        docker {
            image 'python:3.11-slim'
            args '--user root'
        }
    }

    environment {
        APP_NAME  = 'shopflow'
        // IMAGE_TAG sera défini dans le stage Build Docker
    }

    stages {
        // ← vos stages ici (parties 2, 3, 4)
    }

    post {
        always {
            echo 'Pipeline terminé'
        }
        success { echo 'Pipeline ShopFlow réussi' }
        failure { echo 'Pipeline ShopFlow échoué' }
    }
}
```

2.2 Stage Install

Ajoutez ce premier stage dans la section `stages {}` :

Stage Install

```
stage('Install') {
    steps {
        sh '''
            pip install --upgrade pip -q
            pip install -r requirements.txt -q
            echo "Dépendances installées"
        '''
    }
}
```

2.3 Stage Lint : Flake8

Ajoutez le stage Lint après Install :

Stage Lint

```
stage('Lint') {
```

```
    steps {
        sh '''
            flake8 app/ \
                --max-line-length=100 \
                --exclude=__init__.py \
                --format=default
        '''
    }
    post {
        failure {
            echo 'Lint échoué — corriger les erreurs PEP8'
        }
    }
}
```

2.4 Stage Unit Tests

```
Stage Unit Tests
stage('Unit Tests') {
    steps {
        sh '''
            pytest tests/unit/ \
                -v \
                -m unit \
                --junitxml=junit-unit.xml \
                --no-cov
        '''
    }
    post {
        always {
            junit 'junit-unit.xml'   // publie les résultats dans Jenkins
        }
    }
}
```

2.5 Stage Integration Tests

```
Stage Integration Tests
stage('Integration Tests') {
    steps {
        sh '''
            pytest tests/integration/ \
                -v \
                -m integration \
                --junitxml=junit-integration.xml \
                --no-cov
        '''
    }
    post {
        always {
            junit 'junit-integration.xml'
        }
    }
}
```

2.6 Stage Coverage

```
Stage Coverage
stage('Coverage') {
    steps {
        sh '''
```

```
            pytest tests/ \
                --cov=app \
                --cov-report=xml:coverage.xml \
                --cov-report=html:htmlcov \
                --cov-report=term-missing \
                --cov-fail-under=80 \
                --junitxml=junit-report.xml
        '''
    }
    post {
        always {
            publishHTML(target: [
                allowMissing: false,
                alwaysLinkToLastBuild: true,
                keepAll: true,
                reportDir: 'htmlcov',
                reportFiles: 'index.html',
                reportName: 'Coverage Report'
            ])
        }
        failure {
            echo 'Coverage < 80% — ajouter des tests'
        }
    }
}
```

Fail Fast : ordre des stages

- L'ordre Lint → Unit Tests → Integration Tests → Coverage est volontaire : Fail Fast.
- Lint d'abord : erreur PEP8 détectée en < 10s sans perdre du temps à lancer pytest.
- Tests unitaires avant intégration : les tests rapides (< 1s) signalent les bugs avant les tests lents.
- Coverage en dernier : génère coverage.xml dont SonarQube a besoin au stage suivant.
- Si Lint échoue, les stages suivants ne s'exécutent pas, économie de ressources CI.

Q2 : Stages Lint, Tests, Coverage

2.1. Créez le Jenkinsfile avec les 5 stages (Install, Lint, Unit Tests, Integration Tests, Coverage). Lancez le pipeline depuis Jenkins et copiez la vue des stages (Stage View).

2.2. Copiez la sortie du stage Unit Tests montrant les tests passés (format PASSED/FAILED avec noms de tests).

2.3. Copiez la sortie du stage Coverage avec le tableau terminal (Name, Stmts, Miss, Cover). Votre coverage total est-il ≥ 80% ?

2.4. Introduisez volontairement une erreur PEP8 dans app/services/pricing.py (ex: ligne de plus de 100 caractères). Relancez le pipeline. Que se passe-t-il au stage Lint ? Copiez l'erreur affichée.

Partie 3 : SonarQube, Bandit et Quality Gate

Dans cette partie, vous ajoutez l'analyse statique (SonarQube + Pylint + Bandit) et le Quality Gate bloquant au Jenkinsfile.

3.1 Stage Static Analysis : Pylint + Bandit

Ajoutez ce stage après Coverage dans votre Jenkinsfile :

```
Stage Static Analysis
stage('Static Analysis') {
    steps {
        sh '''
            # Pylint — code smells et conventions
            pylint app/ \
                --output-format=parseable \
                --exit-zero \
                > pylint-report.txt || true
            echo "Pylint terminé — voir pylint-report.txt"

            # Bandit — analyse sécurité Python (CWE/OWASP)
            bandit -r app/ \
                -f json \
                -o bandit-report.json \
                --exit-zero

            # Échouer si vulnérabilités HIGH détectées
            python3 -c "
import json, sys
data = json.load(open('bandit-report.json'))
high = [r for r in data.get('results',[])
        if r['issue_severity'] == 'HIGH']
if high:
    print(f'BANDIT: {len(high)} vuln HIGH détectée(s)')
    sys.exit(1)
print('BANDIT: aucune vulnérabilité HIGH')
"
        '''
    }
}
```

3.2 Stage SonarQube Analysis

```
Stage SonarQube Analysis
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {   // nom configuré dans Jenkins
            sh '''
                docker run --rm \
                    --network host \
                    -v $(pwd):/usr/src \
                    -e SONAR_HOST_URL=${SONAR_HOST_URL} \
                    -e SONAR_TOKEN=${SONAR_TOKEN} \
                    sonarsource/sonar-scanner-cli \
                    sonar-scanner \
                        -Dsonar.projectKey=shopflow \
                        -Dsonar.sources=app \
                        -Dsonar.tests=tests \
                        -Dsonar.python.coverage.reportPaths=coverage.xml \
                        -Dsonar.python.pylint.reportPaths=pylint-report.txt
            '''
        }
    }
```

```
}
```

3.3 Stage Quality Gate

```
Stage Quality Gate
stage('Quality Gate') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
            // ↑ bloque si :
            //   - coverage < 80%
            //   - bugs > 0
            //   - vulnerabilities > 0
            //   - code smells > seuil
        }
    }
}
```

Points de vigilance SonarQube

- --network host : obligatoire pour que sonar-scanner-cli atteigne sonarqube:9000 depuis un conteneur Docker.
- coverage.xml doit être généré par le stage Coverage AVANT ce stage : l'ordre des stages est crucial.
- sonar.qualitygate.wait=true dans sonar-project.properties force waitForQualityGate à attendre le résultat.
- En cas de timeout (Quality Gate pending) : augmenter timeout(time: 10) ou vérifier les logs SonarQube.
- Si le Quality Gate est configuré trop strictement, ajuster dans SonarQube → Quality Gates → Conditions.

Q3 : SonarQube, Bandit & Quality Gate

3.1. Ajoutez les stages Static Analysis, SonarQube Analysis et Quality Gate. Relancez le pipeline. Copiez la vue des 8 stages dans Jenkins.

3.2. Accédez à http://localhost:9000/dashboard?id=shopflow. Copiez ou décrivez les métriques affichées (Coverage, Bugs, Code Smells, Vulnerabilities). Le Quality Gate est-il PASSED ou FAILED ?

3.3. Bandit détecte les vulnérabilités selon la nomenclature CWE. Ouvrez bandit-report.json. Listez les 3 premiers résultats trouvés (issue_text + severity). Comment corriger l'un d'eux ?

Partie 4 : Déploiement Staging

Ajoutez les deux derniers stages : build de l'image Docker taguée avec le SHA Git, et déploiement sur l'environnement staging (uniquement sur la branche main).

4.1 Stage Build Docker

```
Stage Build Docker
stage('Build Docker') {
    steps {
        script {
            // TAG = SHA Git court = traçabilité parfaite
            env.IMAGE_TAG = sh(
                script: 'git rev-parse --short HEAD',
                returnStdout: true
            ).trim()
            echo "Build image shopflow:${env.IMAGE_TAG}"
            sh "docker build -t shopflow:${env.IMAGE_TAG} ."
        }
    }
}
```

4.2 Stage Deploy Staging

Créez d'abord le fichier `docker-compose.staging.yml` :

```
docker-compose.staging.yml
# docker-compose.staging.yml
version: '3.9'
services:
  api:
    image: shopflow:${IMAGE_TAG:-latest}   # image buildée au stage précédent
    container_name: shopflow-staging
    ports:
      - "8001:8000"   # port staging différent du port dev
    environment:
      - DATABASE_URL=sqlite:///./shopflow_staging.db
      - REDIS_URL=redis://redis-staging:6379
    depends_on:
      redis-staging:
        condition: service_healthy
    restart: unless-stopped
  redis-staging:
    image: redis:7-alpine
    container_name: shopflow-redis-staging
    healthcheck:
      test: ["CMD","redis-cli","ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    volumes:
      - redis_staging:/data
volumes:
  redis_staging:
```

```
Stage Deploy Staging
stage('Deploy Staging') {
    when {
        branch 'main'   // uniquement sur la branche main
    }
    steps {
        sh '''
```

```
            export IMAGE_TAG=${IMAGE_TAG}
            docker compose -f docker-compose.staging.yml pull || true
            docker compose -f docker-compose.staging.yml up -d --remove-orphans
            docker compose -f docker-compose.staging.yml ps
            # Smoke test sur staging
            sleep 5
            curl -f http://localhost:8001/health || exit 1
            echo "Staging déployé et opérationnel"
        '''
    }
    post {
        failure {
            sh 'docker compose -f docker-compose.staging.yml logs --tail=50'
        }
    }
}
```

Q4 : Build Docker & Deploy Staging

4.1. Ajoutez les stages Build Docker et Deploy Staging. Relancez le pipeline sur la branche main. Copiez la vue complète des 9 stages (pipeline vert).

4.2. Vérifiez que le staging est actif : curl http://localhost:8001/health doit retourner {"status": "ok"}. Copiez la réponse.

4.3. Créez une branche feature/test et poussez un commit. Le stage Deploy Staging s'exécute-t-il ? Pourquoi ? Expliquez le rôle de when { branch 'main' }.

4.4. Expliquez pourquoi on utilise le SHA Git (git rev-parse --short HEAD) comme tag Docker plutôt que latest ou v0.1.0. Quel avantage pour un rollback ?

Partie 5 : Webhook et Déclenchement Automatique

Dans cette partie, vous configurez le déclenchement automatique du pipeline à chaque commit (webhook GitHub/GitLab) et améliorez le post{} du pipeline.

5.1 Configurer le webhook dans Jenkins

Configuration du projet Jenkins

```
# Dans Jenkins — créer un Pipeline projet :
Jenkins → New Item → Pipeline → OK

# Onglet General :
GitHub project : https://github.com/VOTRE_PSEUDO/shopflow

# Onglet Build Triggers :
GitHub hook trigger for GITScm polling
# ou : Poll SCM → Schedule : H/5 * * * *  (toutes les 5 min)

# Onglet Pipeline :
Definition : Pipeline script from SCM
SCM        : Git
URL        : https://github.com/VOTRE_PSEUDO/shopflow.git
Script Path: Jenkinsfile
```

GitHub Webhook

```
# Dans GitHub — ajouter le webhook :
Settings → Webhooks → Add webhook
Payload URL : http://VOTRE_IP:8080/github-webhook/
Content type: application/json
Just the push event

# Si Jenkins n'est pas accessible depuis internet (local),
# utiliser ngrok pour exposer le port 8080 temporairement :
ngrok http 8080
# URL ngrok → utiliser comme Payload URL
```

5.2 Améliorer le bloc post{}

Mettez à jour le bloc `post{}` du Jenkinsfile pour archiver les artefacts et nettoyer :

Bloc post{} complet

```
post {
    always {
        // Archiver les rapports pour consultation dans Jenkins
        archiveArtifacts artifacts: 'junit-*.xml,coverage.xml,bandit-report.json',
                         allowEmptyArchive: true
        // Nettoyer les conteneurs Docker dangling
        sh 'docker system prune -f --filter label=stage=ci || true'
    }
    success {
        echo "Pipeline OK — ShopFlow:${env.IMAGE_TAG} déployé"
    }
    failure {
        echo "Pipeline FAILED — voir les logs ci-dessus"
        // En production : envoyer un email ou une notification Slack
        // mail to:'equipe@shopflow.io', subject:"Pipeline FAILED #${env.BUILD_NUMBER}"
    }
    unstable {
        echo 'Pipeline instable — des tests ont échoué'
    }
}
```

5.3 Jenkinsfile final - récapitulatif des 9 stages

	Stage	Outil	Artefact produit
1	**Install**	`pip`	dépendances installées
2	**Lint**	`flake8`	erreurs PEP8 → fail fast
3	**Unit Tests**	`pytest -m unit`	junit-unit.xml
4	**Integration Tests**	`pytest -m integration`	junit-integration.xml
5	**Coverage**	`pytest --cov`	coverage.xml + htmlcov/
6	**Static Analysis**	`pylint + bandit`	pylint-report.txt + bandit-report.json
7	**SonarQube Analysis**	`sonar-scanner`	rapport SonarQube dashboard
8	**Quality Gate**	`waitForQualityGate`	PASSED/FAILED → bloque si KO
9	**Build + Deploy**	`docker build + compose`	image taguée + staging actif

Q5 : Webhook & Pipeline complet

5.1. Configurez le projet Jenkins et le webhook GitHub. Faites un commit sur main. Vérifiez que Jenkins démarre automatiquement le pipeline. Copiez la log de déclenchement (Build triggered by...).

5.2. Copiez la vue finale du pipeline avec les 9 stages tous en vert (screenshot ou description textuelle stage par stage).